Débutez avec le framework Django

Gestion d’une Liste de Contacts

Python avancé
3 génie informatique
AU 25-26
Dr. Ing. Rihab LAAJILI

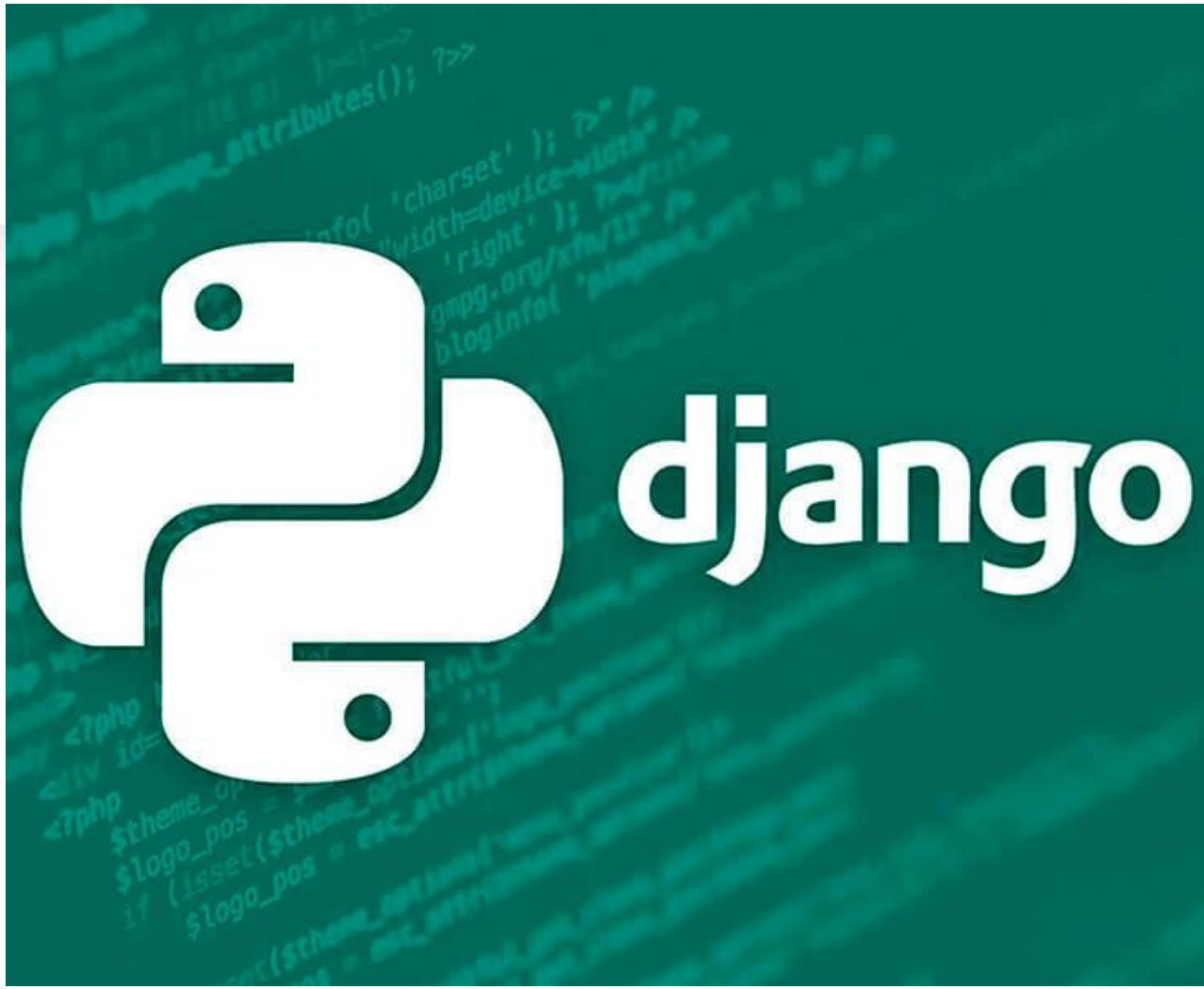
Débutez avec le framework Django

Objectifs pédagogiques

- À la fin de ce cours, vous serez en mesure de :
  - Poser les bases d’une application Django en utilisant des modèles, des vues et des gabarits (templates).
  - Gérer des données stockées dans une base de données à l'aide du site d'administration de Django.
  - Construire une interface CRUD dans Django

Django c'est quoi?

- **Django** est un **framework python open-source** consacré au développement web 2.0 .
- Les concepteurs de Django lui ont attribué le slogan suivant: " *Le framework web pour les perfectionnistes sous pression* ". Il est donc clairement orienté pour les développeurs ayant comme besoin de produire un projet solide rapidement et sans surprise ...
- Pour la petite histoire Django a vu le jour en 2003 et a été publié sous licence BSD en juillet 2005.



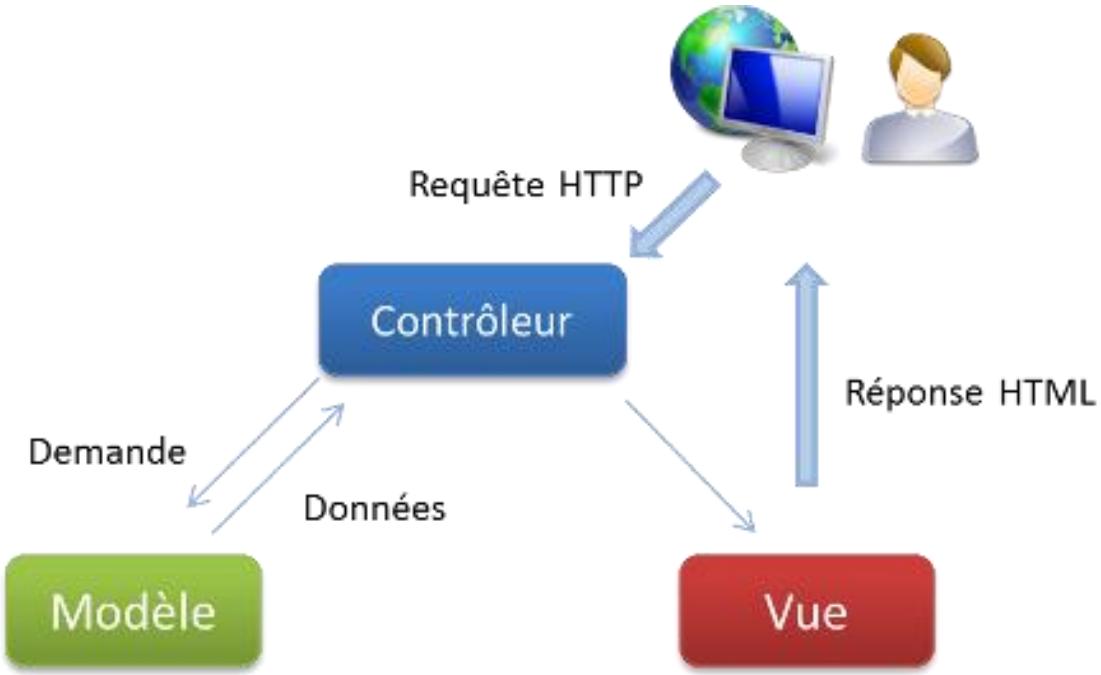
Qui utilise Django?

- Django est apprécié également des grandes entreprises telles que:
  - **Pinterest, Instagram, Libération, 20 minutes, Mozilla,** etc.

- Documentation officielle:
  - https://www.djangoproject.com/

Un peu de théorie : l'architecture MVC

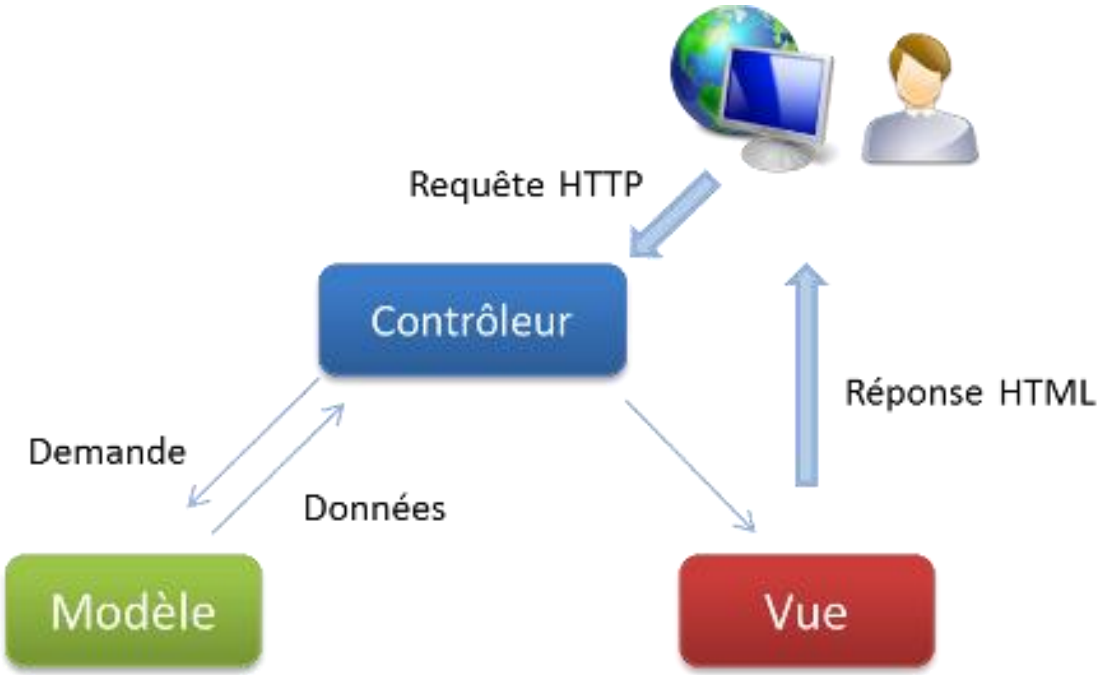
- Architecture utilisée pour structurer les applications avec une interface graphique (ex : pages web dans Django).
- Composée de 3 entités : **Modèle**, **Vue**, et **Contrôleur**.



Un peu de théorie : l'architecture MVC

Le Modèle (Model)

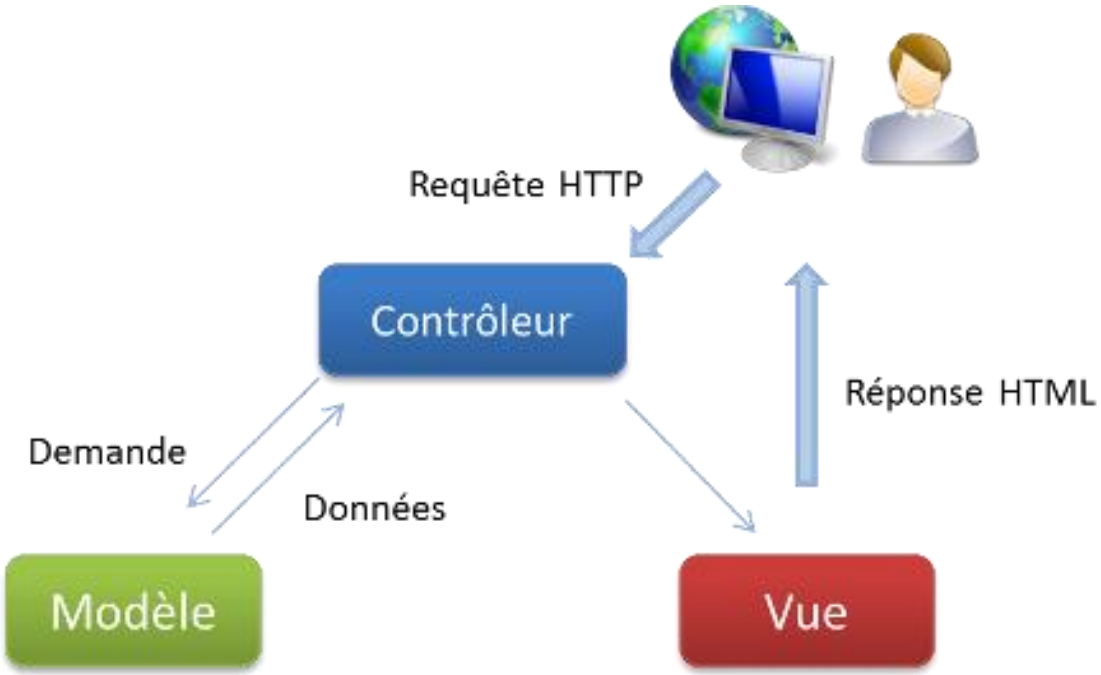
- **Rôle :** Représente et gère les données.
- **Fonctionnalités :**
  - Accès aux informations stockées (souvent dans une base de données).
  - Modification, ajout, suppression, et mise à jour des données.
  - Assure l'intégrité des données (validation et vérification).
  - Interface simplifiée pour interagir avec la base de données.



Un peu de théorie : l'architecture MVC

La Vue (View)

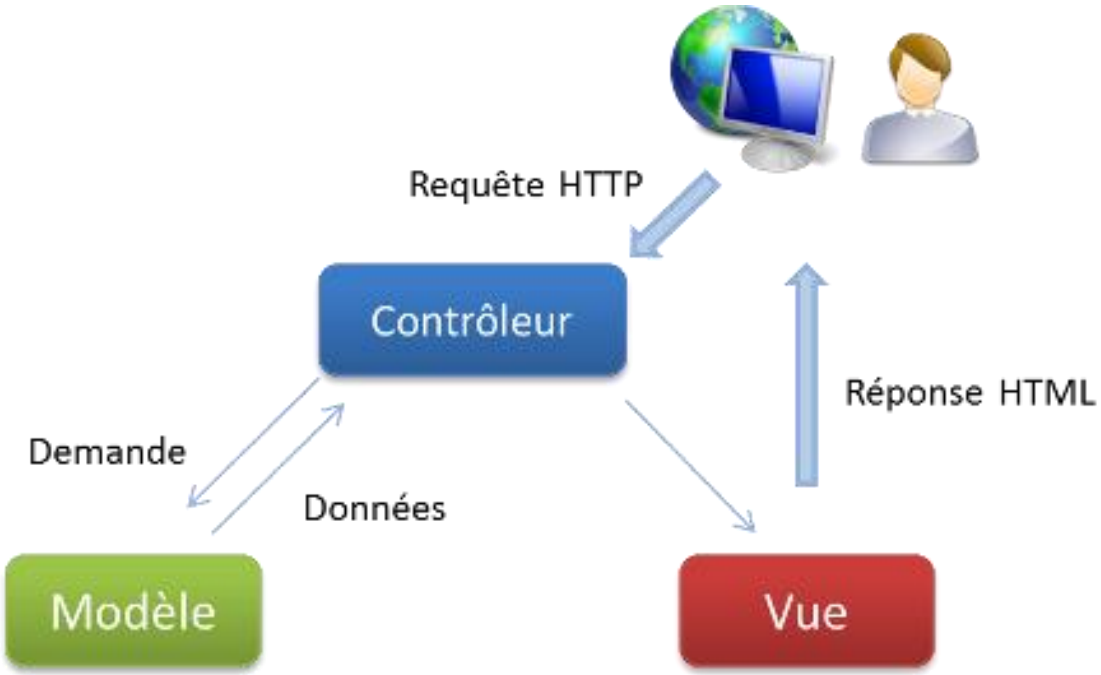
- **Rôle :** Représente la visualisation des données.
- **Fonctionnalités :**
  - Affiche les données à l'utilisateur.
  - Permet à l'utilisateur d'interagir (clics, soumissions de formulaires).
  - Exemple : une page web.



Un peu de théorie : l'architecture MVC

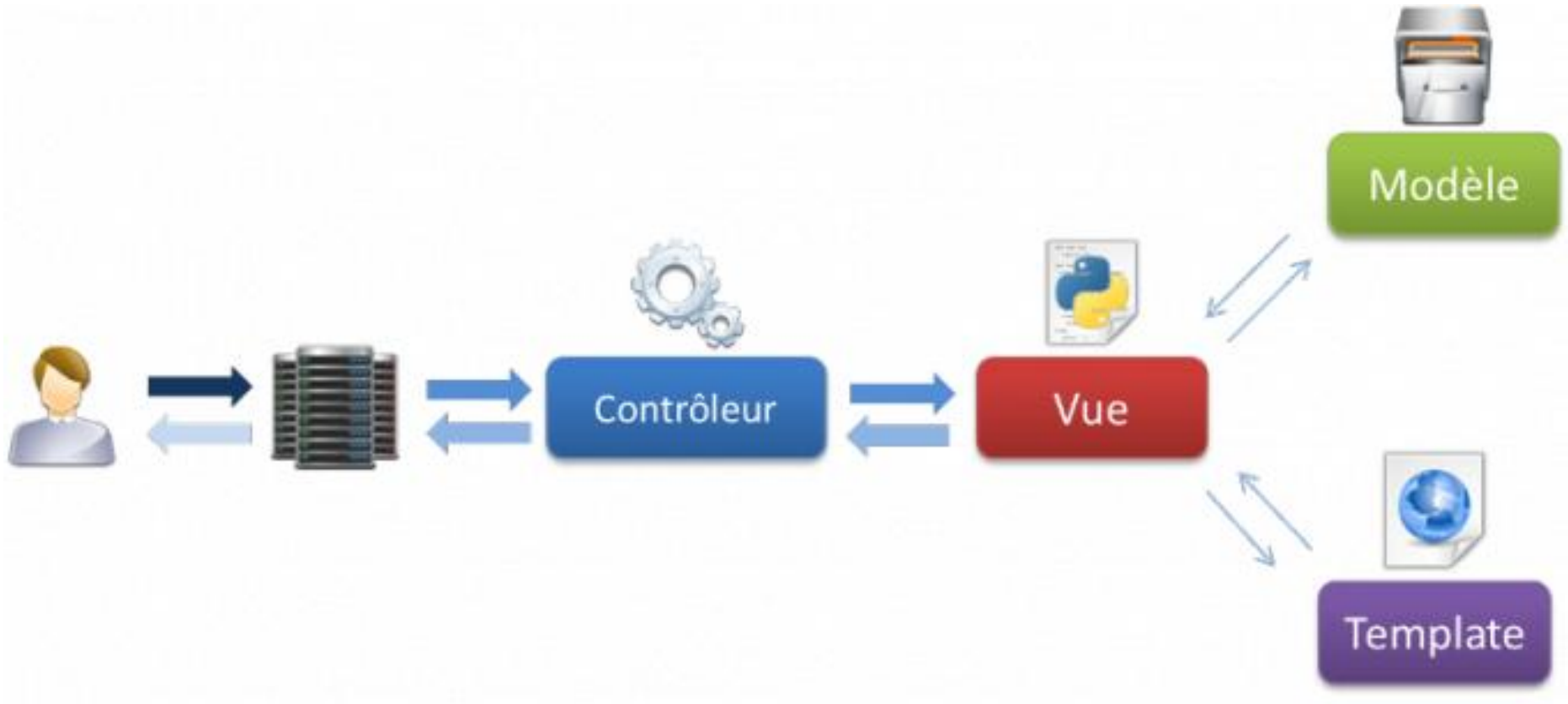
Le Contrôleur (Controller)

- **Rôle :** Gère les interactions de l'utilisateur.
- **Fonctionnalités :**
  - Reçoit les événements utilisateur (clics, soumissions de formulaires).
  - Récupère les données nécessaires depuis le modèle.
  - Transmet les données traitées à la vue pour affichage.
  - Exemple : Lors de l'accès à une page, c'est le contrôleur qui est d'abord sollicité pour déterminer ce qui doit être affiché.



La spécificité de Django : le modèle MVT

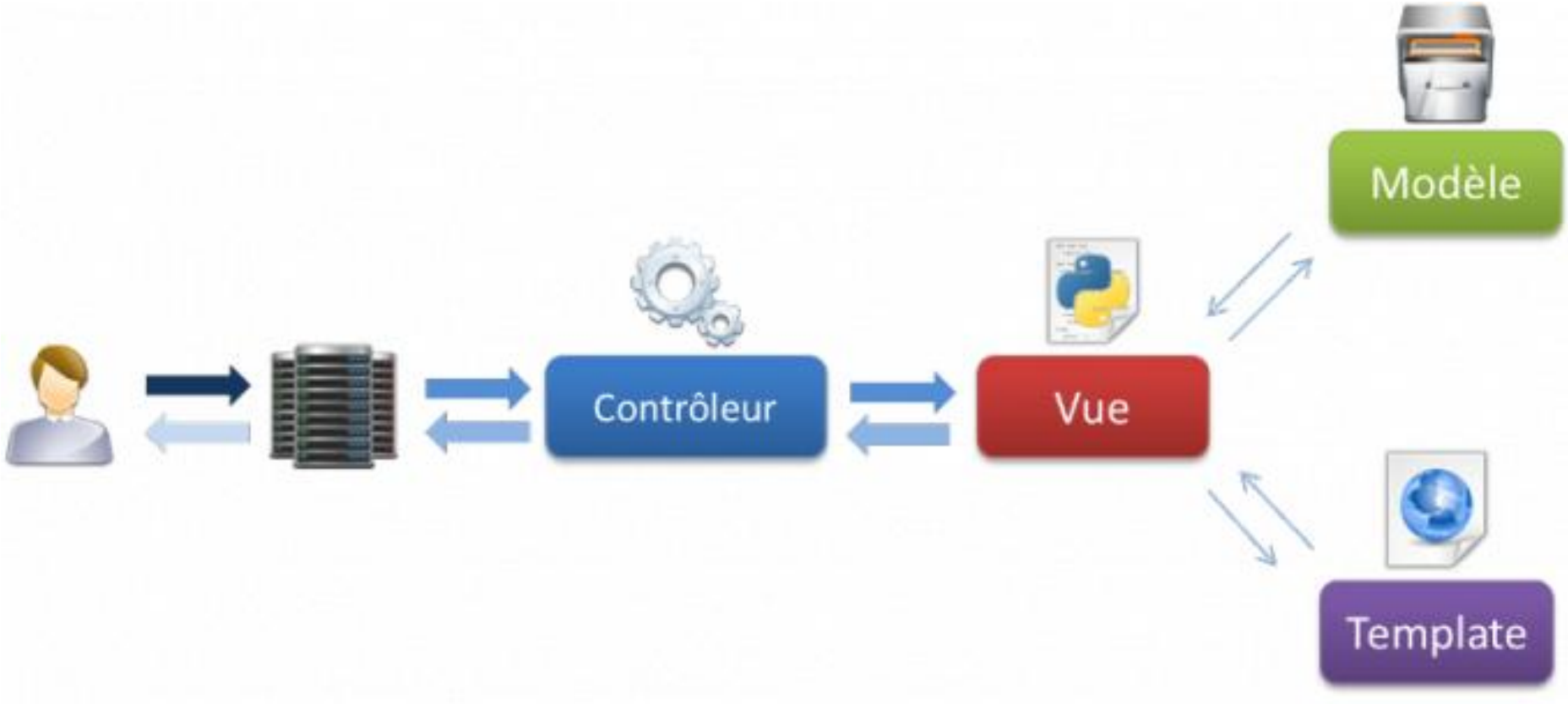
- **Différence avec l'architecture MVC :**
  - Django utilise l'architecture **MVT** (Modèle-Vue-Template) au lieu de l'architecture classique MVC.
  - La principale différence réside dans le fait que Django gère lui-même la partie **Contrôleur**.



La spécificité de Django : le modèle MVT

Le Modèle (Model)

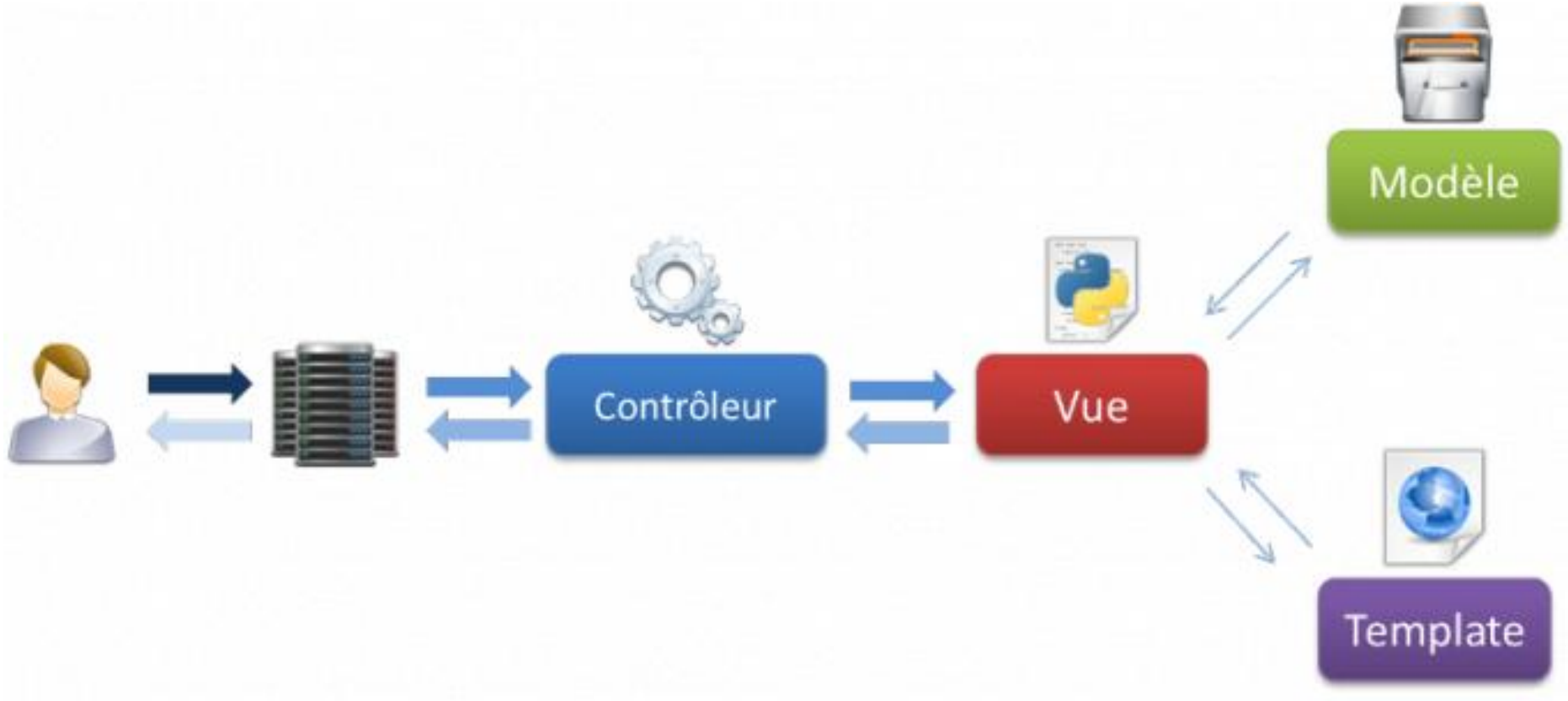
- **Rôle :** Gère les données, tout comme dans l'architecture MVC.
- **Fonctionnalités :**
  - Interaction avec la base de données (accès, modification, validation des données).



La spécificité de Django : le modèle MVT

La Vue (View)

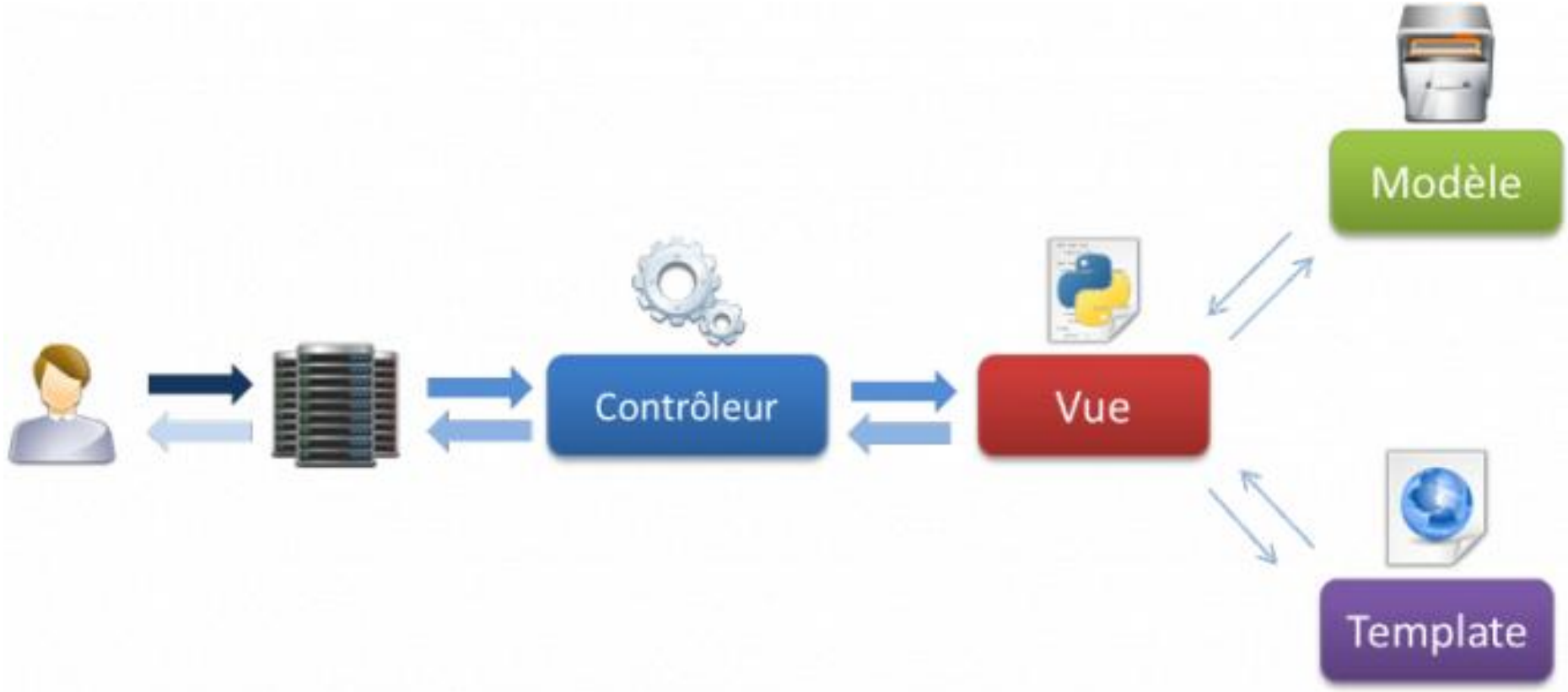
- **Rôle :** Représente la visualisation des données, comme dans MVC.
- **Fonctionnalités :**
  - Affiche les données à l'utilisateur.
  - Reçoit les requêtes du client, les traite et passe les données à un **Template** pour affichage.



La spécificité de Django : le modèle MVT

Le Template (Template)

- **Rôle :** Remplace la vue classique de l'architecture MVC.
- **Fonctionnalités :**
  - Fichier HTML appelé "gabarit".
  - Représente la structure de la page web à afficher.
  - Avant d'être envoyé à l'utilisateur, il est **analysé et exécuté** par Django.
  - Permet d'afficher des variables, utiliser des conditions (if/else), des boucles (for), etc.



Fonctionnement concret de Django

- **Processus général lors de l'appel d'une page :** L'internaute appelle une page du site.
  - **Django** se charge de traiter la requête via les règles de routage URL définies.
  - La **vue** correspondante est exécutée.
  - La vue récupère les données des **modèles** et génère un rendu HTML à partir du **template**.
  - Le serveur renvoie le résultat au navigateur de l'internaute.

Quatre parties essentielles à gérer pour un développeur

1. **Routage des requêtes :**
   - En fonction de l'URL, Django dirige la requête vers la bonne vue.
2. **Modèles (Models) :**
   - Gestion des données (ajout, édition, suppression…).
   - Modèles définis comme des classes avec des attributs correspondant aux champs de la base de données.
3. **Templates :**
   - Affichage des données et autres informations sous forme de pages HTML.
   - Contient la structure de la page à afficher à l'utilisateur.
4. **Vues (Views) :**
   - Récupère les données des modèles et génère un template à partir de celles-ci.
   - Associe la vue à une URL spécifique pour rendre la page accessible.

Lien entre les parties (Modèle, Vue, Template)

- **Modèle (Model) :**
  - Décrit les données sous forme de classe.
  - Chaque attribut de la classe correspond à un champ de la table de la base de données.
  - Django crée automatiquement la table et gère la liaison entre la base de données et les objets de la classe.
  - Aucune requête SQL manuelle n'est nécessaire pour interagir avec la base de données.
- **Vue (View) :**
  - Fonction qui prend en paramètre la requête et les paramètres de l'URL.
  - Exemple : récupère le titre d'un article depuis l'URL (ex : /blog/faire-de-bonnes-crepes), recherche l'article dans la base de données, puis génère le template avec l'article correspondant.

Projets et applications

- **Structure des projets et applications dans Django:**
  - Chaque site Django est un **projet**, composé de **plusieurs applications**.
  - Un projet contient plusieurs applications qui sont responsables de différentes parties du site.
- **Qu'est-ce qu'une application Django ?**
  - Une **application** est un dossier contenant des fichiers relatifs à une tâche spécifique du modèle MVT.
  - Chaque application gère son propre **modèle**, **vue**, **template**, et **schéma d'URL**.
- **Avantages de la séparation en applications :**
  1. **Structure claire du code :**
     - Les **modèles** et **templates** d'une application sont utilisés uniquement dans cette application, évitant les conflits et le mélange de code.
  2. **Réutilisabilité des applications :**
     - Une application bien conçue peut être **facilement réutilisée** dans d'autres projets.
     - Il suffit de copier-coller l'application pour l'intégrer dans un autre projet Django.

En résumé

- Django respecte l'architecture MVT, directement inspirée du très populaire modèle MVC ;
- Django gère de façon autonome la réception des requêtes et l'envoi des réponses au client (partie contrôleur) ;
- Un projet est divisé en plusieurs applications, ayant chacune un ensemble de vues, de modèles et de schémas d'URL ;
- Si elles sont bien conçues, ces applications sont réutilisables dans d'autres projets, puisque chaque application est indépendante.

On commence !

L'environnement virtuel en Python

✅ Un **environnement virtuel** est un conteneur isolé pour gérer les dépendances Python d'un projet.

✅ Il permet d'installer **des paquets spécifiques** sans affecter le système global.

✅ **Pourquoi utiliser un environnement virtuel ?**

- Gérer plusieurs projets avec **des versions différentes** de Python/Django.
- Assurer la **compatibilité des dépendances** et éviter les conflits entre projets.
- Travailler avec **d'anciennes versions** de Python/Django si nécessaire.

✅ **Exemple**

- Un projet peut utiliser **Django 2.0**, tandis qu'un autre utilise **Django 3.0** sur le même ordinateur.

🛠 **Note :** Il existe **plusieurs façons** de créer un environnement virtuel, mais nous allons voir **une méthode spécifique** sur ce cours.

Étapes de la création d'un environnement virtuel

1. Vous pouvez créer un nouveau répertoire nommé "projet-blog" en utilisant la commande "mkdir" dans votre bureau.

```
C:\Users\rihab> cd Desktop

C:\Users\rihab\Desktop>
```

Étapes de la création d'un environnement virtuel

1. Vous pouvez créer un nouveau répertoire nommé "projet-blog" en utilisant la commande "mkdir" dans votre bureau.
2. Changez le répertoire en "projet-blog" en utilisant la commande "cd".

```
C:\Users\rihab\Desktop> mkdir project-blog

C:\Users\rihab\Desktop>cd project-blog

C:\Users\rihab\Desktop\project-blog>
```

Étapes de la création d'un environnement virtuel

3. L'environnement virtuel est créé en utilisant 'python -m venv env', où env est notre environnement virtuel montré par la commande ‘dir'.

```
C:\Users\rihab\Desktop\project-blog>python -m venv env

C:\Users\rihab\Desktop\project-blog>dir
 Le volume dans le lecteur C s’appelle Windows
 Le numéro de série du volume est 0272-3EFB

 Répertoire de C:\Users\rihab\Desktop\project-blog

09/03/2025  23:49    <DIR>          .
09/03/2025  23:43    <DIR>          ..
09/03/2025  23:49    <DIR>          env
               0 fichier(s)                0 octets
               3 Rép(s)  360 818 892 800 octets libres
```

Problème : python -m venv env ne fonctionne pas

Pourquoi ce problème se produit-il ?

1. **Python n'est pas ajouté à la variable d'environnement PATH** ❌
2. **Le module venv n'est pas installé** ❌
3. **Vous utilisez cmd sans les bons privilèges (admin)** ❌

Problème : python -m venv env ne fonctionne pas

1️⃣ Vérifier si Python est installé et accessible

Tapez cette commande pour vérifier si Python est bien installé :

```sh
python --version
```

Si une erreur apparaît :

✔ **Solution :** Ajouter Python à la variable d'environnement `PATH` (voir étape 3).

Problème : python -m venv env ne fonctionne pas

2️⃣ Vérifier si `venv` est disponible

Tapez :

```sh
python -m ensurepip --default-pip
```

Si cela ne fonctionne pas, vous devrez peut-être réinstaller Python avec `venv` inclus.

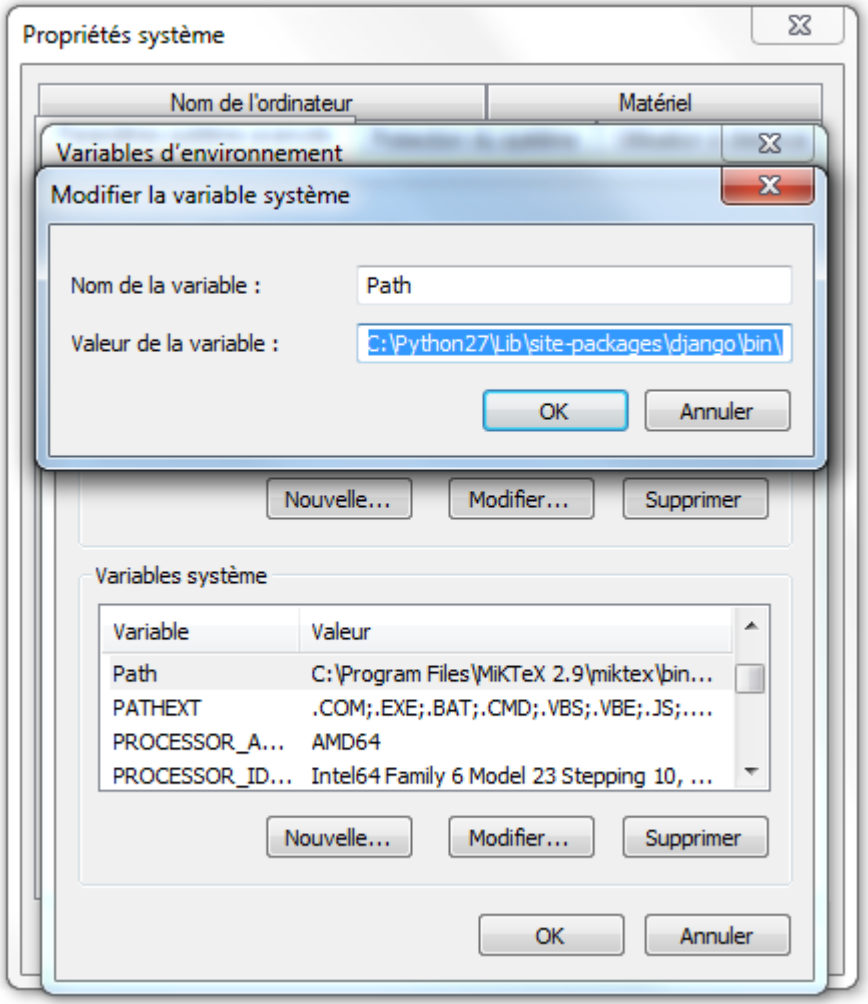
3 Ajouter Python à la variable d'environnement `PATH`

Si Python n'est pas reconnu, il faut l'ajouter manuellement au `PATH` :

- **Étapes :**

1. **Ouvrir les Paramètres Windows** (`Win + I`).
2. Aller dans **Système → À propos → Paramètres avancés du système**.
3. Dans l'onglet **Avancé**, cliquer sur **Variables d'environnement**.
4. Dans **Variables système**, chercher **Path** et cliquer sur **Modifier**.
5. Ajouter les chemins suivants (en adaptant avec ta version) :
   - `C:\Users\rihab\AppData\Local\Programs\Python\PythonXX\`
   - `C:\Users\rihab\AppData\Local\Programs\Python\PythonXX\Scripts\`
6. **Redémarrer** `cmd` et réessayer :

```sh
python -m venv env
```



Problème : python -m venv env ne fonctionne pas

4️⃣ Exécuter `cmd` en mode administrateur

Si vous avez toujours des erreurs, essayez d'ouvrir `cmd` en mode **Administrateur** :

- Cherchez `cmd` dans le menu Démarrer.
- Faites **clic droit → Exécuter en tant qu'administrateur**.

Étapes de la création d'un environnement virtuel

4. Pour activer votre environnement virtuel : L'environnement virtuel peut être activé à l'aide de la commande "source", où le dossier "Scripts" doit être activé.

```
C:\Users\rihab\Desktop\project-blog>env\Scripts\activate

(env) C:\Users\rihab\Desktop\project-blog>
```

Étapes de la création d'un environnement virtuel

5. Installation de Django : Vous pouvez utiliser 'pip install django' pour installer Django dans votre environnement virtuel spécifique.

```
(env) C:\Users\rihab\Desktop\project-blog>pip install django
```

Création d'un projet Django

1. La première étape consiste à créer votre projet en utilisant la commande 'django-admin startproject project_name', où 'project_name' est 'django_blog' dans votre cas.

- Il générera de nombreux fichiers dans notre projet nouvellement créé, que vous pourrez rechercher plus en détail dans la documentation de Django si nécessaire.

```
(env) C:\Users\rihab\Desktop\project-blog>django-admin startproject crepes_bretonnes
```

Création d'un projet Django

2. Changez le répertoire du projet nouvellement créé à l'aide de la commande "cd" et affichez le fichier créé à l'aide de la commande ‘dir’.

```
(env) C:\Users\rihab\Desktop\project-blog>cd crepes_bretonnes

(env) C:\Users\rihab\Desktop\project-blog\crepes_bretonnes>dir
 Le volume dans le lecteur C s’appelle Windows
 Le numéro de série du volume est 0272-3EFB

 Répertoire de C:\Users\rihab\Desktop\project-blog\crepes_bretonnes

10/03/2025  00:52    <DIR>          .
10/03/2025  00:52    <DIR>          ..
10/03/2025  00:52    <DIR>          crepes_bretonnes
10/03/2025  00:52               694 manage.py
               1 fichier(s)              694 octets
               3 Rép(s)  361 122 222 080 octets libres

(env) C:\Users\rihab\Desktop\project-blog\crepes_bretonnes>
```

Création d'un projet Django

Un nouveau dossier nommé crepes_bretonnes est apparu et possède la structure suivante :

```
crepes_bretonnes/
        manage.py
        crepes_bretonnes/
                __init__.py
                settings.py
                urls.py
                wsgi.py
```

Structure d'un projet Django :

•**Le projet Django** : Par exemple, dans le projet **crepes_bretonnes**, nous avons deux éléments principaux :

- **manage.py**
- Un sous-dossier nommé également **crepes_bretonnes**

Création d'un projet Django

Un nouveau dossier nommé crepes_bretonnes est apparu et possède la structure suivante :

```
crepes_bretonnes/
        manage.py
        crepes_bretonnes/
                __init__.py
                settings.py
                urls.py
                wsgi.py
```

Contenu du sous-dossier crepes_bretonnes :

•Quatre fichiers Python :

- **settings.py** : Contient la configuration du projet.
- **urls.py** : Rassemble les URL du site et les fonctions à appeler pour chaque URL.
- **wsgi.py** : Fichier de configuration pour le serveur web.
- **init.py** : Indique que ce dossier est un package Python.

Création d'un projet Django

Un nouveau dossier nommé crepes_bretonnes est apparu et possède la structure suivante :

```
crepes_bretonnes/
        manage.py
        crepes_bretonnes/
                __init__.py
                settings.py
                urls.py
                wsgi.py
```

Créer un dossier "templates" :Dans le dossier principal crepes_bretonnes, créez un dossier "templates" qui contiendra vos fichiers HTML.

Création d'un projet Django

Un nouveau dossier nommé crepes_bretonnes est apparu et possède la structure suivante :

```
crepes_bretonnes/
        manage.py
        crepes_bretonnes/
                __init__.py
                settings.py
                urls.py
                wsgi.py
```

À ne pas modifier :

•Ne touchez pas aux fichiers **wsgi.py** et **init.py** : ils ne sont pas destinés à être modifiés.

Création d'un projet Django

Un nouveau dossier nommé crepes_bretonnes est apparu et possède la structure suivante :

```
crepes_bretonnes/
        manage.py
        crepes_bretonnes/
                __init__.py
                settings.py
                urls.py
                wsgi.py
```

Le fichier manage.py :

•**manage.py** est un raccourci local de la commande **django-admin.py**.

•Il prend en charge la configuration et l'administration de votre projet.

•Utilisez **manage.py** pour gérer le projet, et oubliez la commande **django-admin.py**, qui ne sert qu'à créer des projets.

Création d'un projet Django

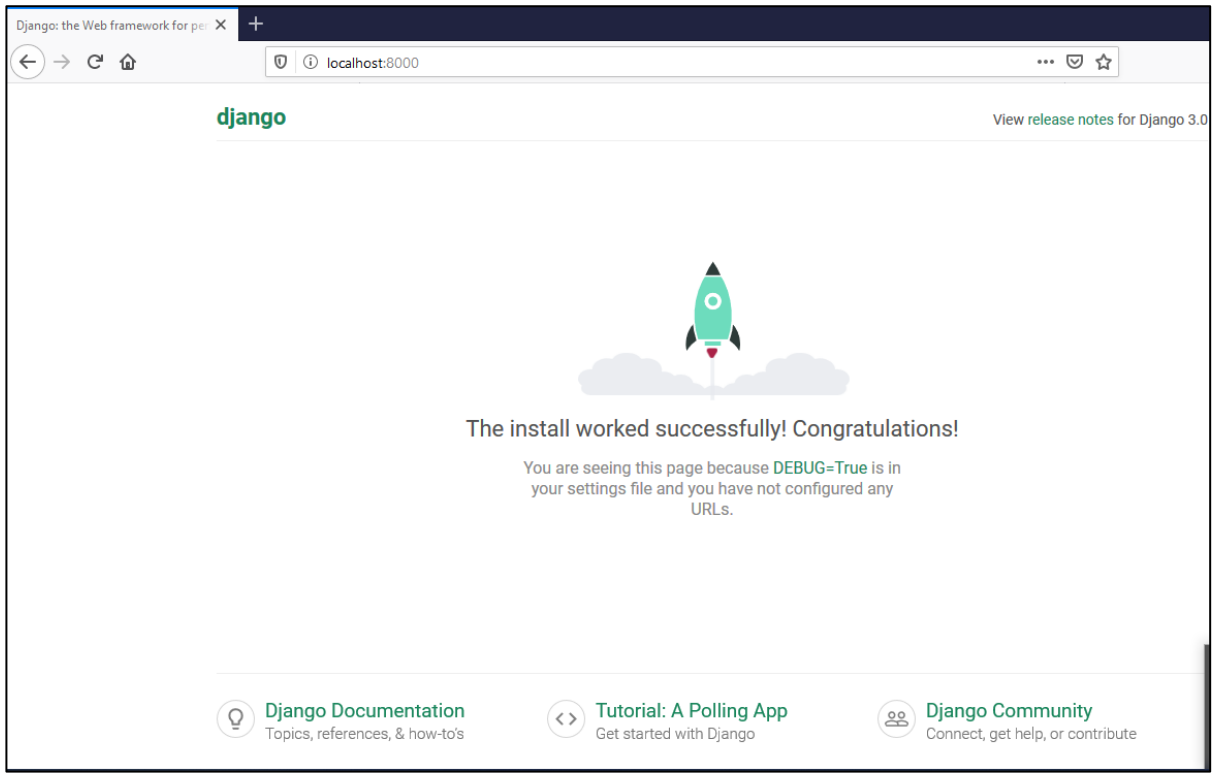
3. Vous pouvez exécuter votre projet en utilisant " python manage.py runserver ".

```
(env) C:\Users\rihab\Desktop\project-blog\django_blog>python manage.py runserver
```

🚀 Cela démarrera le serveur à l’adresse http://127.0.0.1:8000/

Création d'un projet Django

4. Le projet peut être visualisé dans votre navigateur préféré (Google Chrome, Mozilla Firefox, etc.). Vous pouvez entrer dans votre navigateur et taper 'localhost:8000' ou '127.0.0.1:8000' dans l'URL



Configurez votre projet

- **Fichier settings.py** : Fichier Python contenant une liste de **variables de configuration** pour votre projet Django.
- Vous pouvez modifier ces variables pour configurer votre projet.
- **Variables importantes dans settings.py :**

1.DEBUG :

- **DEBUG = True** : Active le mode de débogage (affiche les erreurs détaillées).
- **DEBUG = False** : Désactive le mode débogage (utilisé en production).

2.SECRET_KEY :

- Utilisé pour la sécurité, **ne pas le partager**.
- Assurez-vous de changer cette clé par une valeur unique et secrète.

3.INSTALLED_APPS :

- Liste des applications Django utilisées dans votre projet.
- Vous y ajouterez les applications que vous créez ou installez.

4.MIDDLEWARE :

- Liste des **middleware** utilisés pour traiter les requêtes HTTP.
- Chaque middleware effectue une tâche spécifique avant ou après une requête.

5.DATABASES :

- Configuration de la base de données.
- Par défaut, Django utilise SQLite, mais vous pouvez configurer d'autres bases de données (comme PostgreSQL, MySQL, etc.).

6.TEMPLATES :

- Configuration des **templates** HTML utilisés par Django.
- Vous définissez où Django doit chercher les fichiers templates et les paramètres associés.

7.STATIC_URL :

- Définition du chemin des fichiers statiques (CSS, JavaScript, images).
- Par défaut : /static/, mais vous pouvez le configurer selon vos besoins.

Configurez votre projet

- **Fichier settings.py** : Fichier Python contenant une liste de **variables de configuration** pour votre projet Django.
- Vous pouvez modifier ces variables pour configurer votre projet.
- **Variables importantes dans settings.py :**

```
DEBUG = True
TEMPLATE_DEBUG = DEBUG
```

- **DEBUG** : **DEBUG = True** :
  - Active le mode de débogage.
  - Affiche des informations détaillées en cas d'erreurs dans le code (utile pendant le développement).
  - Ces informations peuvent inclure des données sensibles (comme des chemins de fichiers et des informations de configuration).
  - **À ne jamais utiliser en production**.
- **DEBUG = False** :
  - Désactive le mode de débogage.
  - N'affiche pas de détails d'erreur sur le site.
  - **Recommandé en production** pour éviter d'exposer des données sensibles.

Configurez votre projet

- **Fichier settings.py** : Fichier Python contenant une liste de **variables de configuration** pour votre projet Django.
- Vous pouvez modifier ces variables pour configurer votre projet.
- **Variables importantes dans settings.py :**

```
ADMINS = (
    ('rihab LAAJILI', 'rihablaajili@gmail.com'),
)
```

•**ADMINS** est un **tuple** qui contient plusieurs sous-tuple(s).
•Chaque **sous-tuple** contient :
1. Le **nom** de l'administrateur ou du gestionnaire (par exemple, 'rihab LAAJILI').
2. L'**adresse e-mail** de l'administrateur (par exemple, 'rihablaajili@gmail.com').

•On l'utilise lorsque le site rencontre une erreur (en mode production), Django peut envoyer des notifications par e-mail aux **administrateurs** listés dans ADMINS.

Configurez votre projet

- **Fichier settings.py** : Fichier Python contenant une liste de **variables de configuration** pour votre projet Django.
- Vous pouvez modifier ces variables pour configurer votre projet.
- **Variables importantes dans settings.py :**

La configuration de la base de données se fait dans le dictionnaire DATABASES.

•**ENGINE** : Indique le backend de la base de données. Dans le cas d'une base de données SQLite, il doit être 'django.db.backends.sqlite3'.

```
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.sqlite3',
    'NAME': 'database.sql',
    'USER': '',
    'PASSWORD': '',
    'HOST': '',
    'PORT': '',
  }
}
```

Avantages de SQLite pour le développement local :

•**Pas besoin de serveur séparé** : SQLite est un moteur de base de données embarqué qui fonctionne en utilisant un fichier local, ce qui signifie que vous n'avez pas besoin de configurer un serveur de base de données comme MySQL ou PostgreSQL.

•**Facilité d'utilisation** : Il suffit d'un fichier pour stocker toutes les données de l'application, et Django gère automatiquement les interactions avec la base de données.

Configurez votre projet

- **Fichier settings.py** : Fichier Python contenant une liste de **variables de configuration** pour votre projet Django.
- Vous pouvez modifier ces variables pour configurer votre projet.
- **Variables importantes dans settings.py :**

```
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.sqlite3',
    'NAME': 'database.sql',
    'USER': '',
    'PASSWORD': '',
    'HOST': '',
    'PORT': '',
  }
}
```

La configuration de la base de données se fait dans le dictionnaire DATABASES.

•**ENGINE** : Indique le backend de la base de données. Dans le cas d'une base de données SQLite, il doit être 'django.db.backends.sqlite3'.
•**NAME** : Le chemin du fichier de la base de données. Django crée automatiquement un fichier db.sqlite3 dans le répertoire de votre projet (BASE_DIR).
•**USER**, **PASSWORD**, **HOST**, **PORT** : Ces options sont laissées vides car elles sont utilisées pour des bases de données **serveur** comme MySQL, PostgreSQL ou Oracle. Pour SQLite, ces paramètres ne sont pas nécessaires.

Configurez votre projet

- **Fichier settings.py** : Fichier Python contenant une liste de **variables de configuration** pour votre projet Django.
- Vous pouvez modifier ces variables pour configurer votre projet.
- **Variables importantes dans settings.py :**

```
TIME_ZONE = 'Europe/Paris'
LANGUAGE_CODE = 'fr-FR'
```

1. Modifier le fuseau horaire :
Le fuseau horaire permet de définir la localisation temporelle de votre projet.
Vous pouvez modifier la ligne de TIME_ZONE pour l'adapter à votre localisation (par exemple, **Europe/Paris** pour la France).
python
2. Modifier le fuseau horaire :
Le fuseau horaire permet de définir la localisation temporelle de votre projet.
Vous pouvez modifier la ligne de TIME_ZONE pour l'adapter à votre localisation (par exemple, **Europe/Paris** pour la France).
python

Configurez votre projet

Configurer les Templates dans Django

Configuration dans settings.py

- **BACKEND** : Spécifie le moteur de templates à utiliser.
  - django.template.backends.django.DjangoTemplates (le moteur de templates par défaut).
- **DIRS** : Liste des répertoires où Django va chercher des templates.
  - Exemple : os.path.join(BASE_DIR, 'templates')
  - Cela permet de faire référence au dossier templates à la racine du projet.
- **APP_DIRS** : Active la recherche dans les dossiers templates de chaque application.
  - Valeur : True
- **OPTIONS** : Contient des **context processors** qui fournissent des informations globales aux templates.
  - Exemple : 'django.contrib.auth.context_processors.auth' pour l'authentification de l'utilisateur.

Configurez votre projet

Configurer les Templates dans Django

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [
            os.path.join(BASE_DIR, 'templates'),  # Le dossier 'templates' à la racine
        ],
        'APP_DIRS': True,  # Permet de chercher dans les dossiers templates des apps
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

```
project_blog/
├── manage.py
├── project_blog/
└── templates/
    ├── base.html
    ├── index.html
    └── autre_template.html
```

Notre première app

1.Créer une Application Django

- Dans le terminal, dans le répertoire principal de ton projet, utilise la commande suivante pour créer une nouvelle application Django :

```
python manage.py startapp hello
```

- Cette commande crée un dossier hello avec tous les fichiers nécessaires à l'application.

2. Créer une Vue (View)

- Ouvre le fichier hello/views.py et crée une vue qui renvoie "Hello, World!".
- Exemple dans hello/views.py :

```
from django.http import HttpResponse


def hello_world(request):
    return HttpResponse("Hello, World!")
```

3. Configurer les URLs de l'Application

- Crée .hello un fichier **urls.py** dans le dossier hello.
- Dans ce fichier, crée un chemin pour la vue.
- Exemple dans hello/urls.py :

```
from django.urls import path
from . import views

urlpatterns = [
    path('hello/', views.hello_world, name='hello_world'),
]
```

3. Configurer les URLs de l'Application

```
from django.urls import path
from . import views

urlpatterns = [
    path('hello/', views.hello_world, name='hello_world'),
]
```

•**from django.urls import path :** Cette ligne importe la fonction path depuis le module django.urls, qui est utilisée pour définir les routes (URL) dans une application Django.

•**from . import views :** Cette ligne importe le module views situé dans le même répertoire que le fichier actuel (indiqué par le .). Le module views contient généralement les fonctions qui traitent la logique des vues pour les différentes URL.

3. Configurer les URLs de l'Application

```
from django.urls import path
from . import views

urlpatterns = [
    path('hello/', views.hello_world, name='hello_world'),
]
```

•**urlpatterns :** Il s'agit d'une liste qui contient toutes les URL définies pour l'application Django. Chaque URL dans cette liste est associée à une vue spécifique, qui sera exécutée lorsque cette URL est visitée.

•**path('hello', view.hello_world, name='Hello_world') :**

- •**'hello' :** C'est la partie de l'URL (chemin) qui sera utilisée pour accéder à cette vue. Lorsque l'utilisateur accède à localhost:8000/hello, la vue hello_world sera appelée.
- •**view.hello_world :** C'est la vue associée à cette URL. Cependant, il y a une erreur ici : tu as écrit view.hello_world, mais tu devrais écrire views.hello_world. Donc, il faut corriger view en views (référence au module views importé juste avant).
- •**name='Hello_world' :** Ce paramètre permet de nommer cette URL. Cela permet de l'utiliser facilement dans des templates ou dans des redirections, comme par exemple : {% url 'Hello_world' %}.

4. Inclure l'URL de l'Application dans le Fichier urls.py du Projet

- Ouvre le fichier urls.py de ton projet Django (généralement situé dans le dossier du projet, par exemple project_blog/urls.py).
- Inclut l'URL de ton application hello.
- Exemple dans project_blog/urls.py :

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('hello/', include('hello.urls')),  # Remplace 'hello' par le nom de ton appli
]
```

4. Inclure l'URL de l'Application dans le Fichier urls.py du Projet

- **Explication des modifications :**

```
from django.urls import path, include
```

- **Avant** : Le code original importait seulement path.
- **Après** : L'importation de include a été ajoutée. La fonction include() est utilisée pour inclure les URL des applications dans le projet principal. Cela permet de séparer la gestion des URL entre différentes applications Django.

4. Inclure l'URL de l'Application dans le Fichier urls.py du Projet

- **Explication des modifications :**

```
path('hello/', include('hello.urls'))
```

- **Avant** : Le code original ne contenait qu'une seule ligne de routage pour l'administration.
- **Après** : Une nouvelle ligne a été ajoutée pour inclure les URL de l'application hello dans le projet. Cela signifie que toutes les URL définies dans l'application hello seront accessibles sous le chemin /hello/. Cette ligne fait référence au fichier urls.py de l'application hello pour définir des chemins supplémentaires à partir de /hello/.

5. Vérifier l'Application dans le Fichier settings.py

- Assure-toi que l'application hello est bien incluse dans le fichier settings.py sous la section INSTALLED_APPS.
- Exemple dans project_blog/settings.py :

```
INSTALLED_APPS = [
    # Autres applications...
    'hello',  # Assurez-vous que votre application 'hello' est incluse ici
]
```

6. Lancer le Serveur Django

- Dans le terminal, lance le serveur de développement pour voir les résultats.

```
python manage.py runserver
```

- Accéder à ‘’Hello, World!’’ : Ouvre un navigateur et accède à l'URL suivante:

http://127.0.0.1:8000/hello/

1. Créer un fichier HTML

créer un fichier template pour ta page dans ton application hello. Voici comment procéder :

1. Crée un dossier **templates** à l'intérieur de ton application hello (si ce n'est pas déjà fait).
2. Crée un fichier **hello_world.html** à l'intérieur de ce dossier **templates/hello/** (le chemin complet sera donc : hello/templates/hello/hello_world.html).

2. Modifier la vue pour utiliser le template

modifier la vue hello_world pour utiliser ce fichier HTML au lieu de retourner simplement un texte avec HttpResponse.
Modifie ton fichier **hello/views.py** pour ressembler à ceci :

```
from django.shortcuts import render


def hello_world(request):
    return render(request, 'hello/hello_world.html')
```

La fonction render de Django permet de rendre un template HTML, et elle passe par défaut le contexte de la requête.

3. Vérifier la configuration des templates dans settings.py

Assure-toi que Django est configuré pour rechercher les templates dans le dossier de ton application. Dans settings.py, vérifie que DIRS dans la configuration TEMPLATES inclut les chemins vers les templates de ton application :

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [
            BASE_DIR / 'templates',  # Tu peux aussi ajouter ici un chemin vers les templates
        ],
        'APP_DIRS': True,  # Cela permet à Django de chercher les templates dans les apps
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

3. Redémarrer le Serveur Django

- Dans le terminal, lance le serveur de développement pour voir les résultats.

```
python manage.py runserver
```

- Accéder à ‘’Hello, World!’’ : Ouvre un navigateur et accède à l'URL suivante:

http://127.0.0.1:8000/hello/

À suivre …

Go watch the movie